



CSE6324-SOFTWARE VERIFICATION & VALIDATION

Topic: Hospital Management System

Student Name	Student ID	Email
CHIN ZHEN HO	1221102540	1221102540@student.mmu.edu.my
PANG TIAN YOU	1231303394	1231303394@student.mmu.edu.my
LIM KAI SHEN	1211110602	1211110602@student.mmu.edu.my

Submission Date: 18/6/2026

1.0 Objective

To ensure comprehensive test coverage and efficient test management for the Hospital Management System (HMS), our group proposes a testing suite utilizing the following three tools: Katalon, Selenium IDE, and TestCase Studio.

2.0 Tool Configuration & Environment

- **Selected Tool:** Katalon Recorder (Chrome Browser Extension)
- **System Under Test (SUT):** Hospital Management System (HMS)
- **Environment:** Local XAMPP Server (Apache & MySQL) via localhost
- **Testing Scope:** Black-box Web UI functional testing.

3.0 Source Code Repository and System Execution Guide

This section explains how the lecturer can access the Hospital Management System (HMS) source code from GitHub and run the system in a local testing environment. The purpose of this section is to ensure that the system under test can be reproduced before executing the proposed automated and manual test cases.

3.1 GitHub Repository Access

The HMS source code can be accessed through the GitHub repository below:

GitHub Repository:

<https://github.com/kishan0725/Hospital-Management-System>

The lecturer may obtain the source code using either of the following methods.

Method 1: Download ZIP from GitHub

1. Open the GitHub repository link in a web browser.
2. Click the green Code button.
3. Select Download ZIP.
4. Extract the downloaded ZIP file.
5. Move the extracted project folder into the XAMPP htdocs directory.

For Windows, the folder should be placed in:

C:\xampp\htdocs\Hospital-Management-System-master

Method 2: Clone the Repository Using Git

If Git is installed, the repository can be cloned using the following command:

git clone https://github.com/kishan0725/Hospital-Management-System.git

After cloning, move or ensure the project folder is located inside the XAMPP htdocs directory.

Example path:

C:\xampp\htdocs\Hospital-Management-System

3.2 Required Software

Before running the HMS, the following software should be installed:

Software / Tool	Purpose
XAMPP	Provides Apache web server and MySQL database server
Google Chrome	Browser used to access and test the HMS
phpMyAdmin	Used to create and import the HMS database
VS Code / Sublime Text	Optional editor for viewing or modifying source code
Katalon Studio / Katalon Recorder	Used for automated functional test execution
Selenium IDE	Used for browser-based automated testing
TestCase Studio	Used to capture manual test steps, test data, locators, and screenshots

3.3 Database Setup

The HMS requires a MySQL database before the system can be executed.

1. Open the XAMPP Control Panel.
2. Start **Apache**.
3. Start **MySQL**.
4. Open a browser and go to: **http://localhost/phpmyadmin**
5. Click **New** on the left panel.
6. Create a new database named: **myhmsdb**
7. Select the newly created **myhmsdb** database.
8. Click the **Import** tab.
9. Choose the SQL file from the project folder: **myhmsdb.sql**
10. Click **Go** to import the database.

After the import is completed, the HMS database tables should be available in phpMyAdmin. The main tables include patient records, doctor records, appointment records, prescription records, contact queries, and admin login records.

3.4 Running the HMS System

After the project folder has been copied into **htdocs** and the database has been imported, the HMS can be launched through a browser.

If the folder name is **Hospital-Management-System-master**, open:

http://localhost/Hospital-Management-System-master/index.php

If the folder name is **Hospital-Management-System**, open:

http://localhost/Hospital-Management-System/index.php

The HMS home page should be displayed. From the home page, users can access the Patient, Doctor, and Admin login sections.

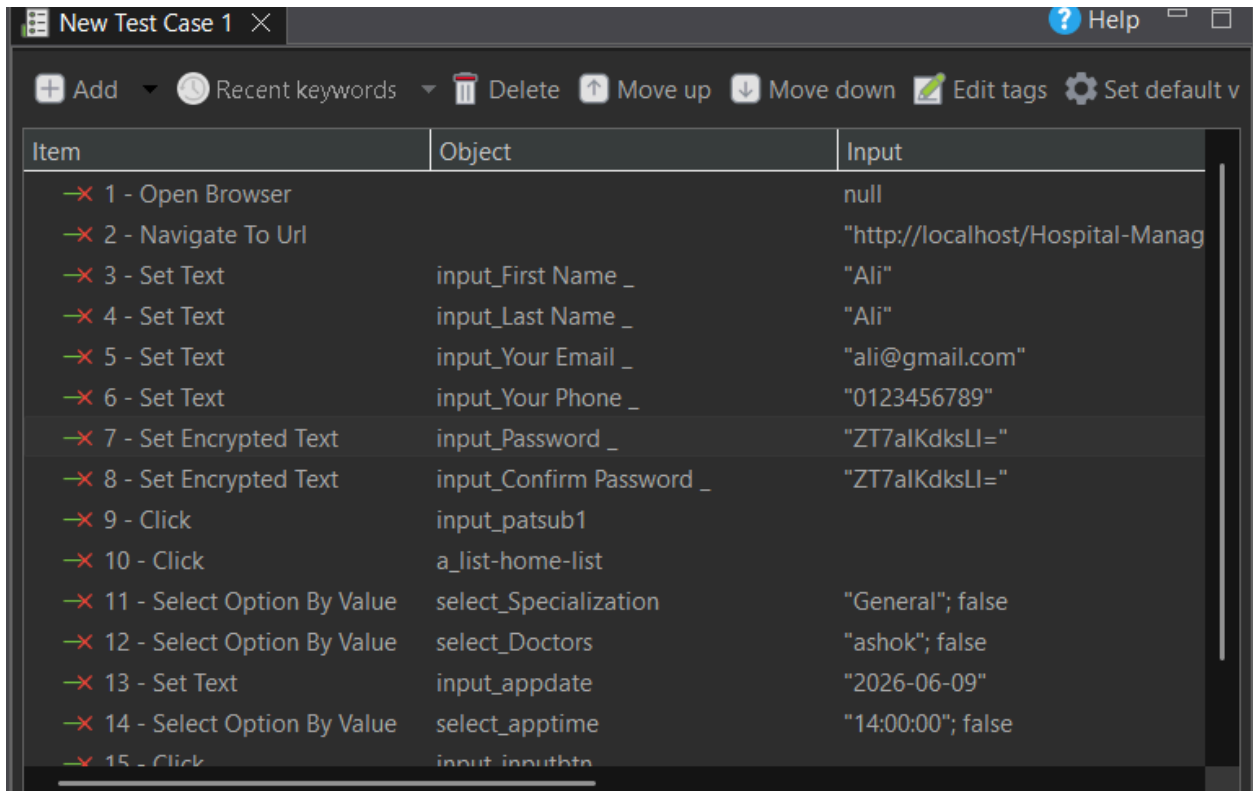
3.5 Default Login Credentials for Testing

The imported database already contains sample user accounts. These accounts can be used by the lecturer to verify and test the system functions.

User Role	Login ID / Email	Password
Admin	admin	admin123
Doctor	ashok	ashok123
Doctor	Dinesh	dinesh123
Patient	alia@gmail.com	alia123
Patient	kishansmart0@gmail.com	kishan123

The admin account can be used to view patient records, doctor records, appointment details, prescription lists, contact queries, and to add or delete doctor records. The doctor account can be used to view appointments and issue prescriptions. The patient account can be used to register, login, book appointments, view appointment history, cancel appointments, and view prescriptions.

Tool 1: Katalon Studio



The screenshot shows the 'New Test Case 1' window in Katalon Studio. The interface includes a toolbar with buttons for 'Add', 'Recent keywords', 'Delete', 'Move up', 'Move down', 'Edit tags', and 'Set default v'. Below the toolbar is a table with three columns: 'Item', 'Object', and 'Input'. The table contains 15 rows of test steps, each with a red 'X' icon in the 'Item' column. The steps are as follows:

Item	Object	Input
1 - Open Browser		null
2 - Navigate To Url		"http://localhost/Hospital-Manag
3 - Set Text	input_First Name _	"Ali"
4 - Set Text	input_Last Name _	"Ali"
5 - Set Text	input_Your Email _	"ali@gmail.com"
6 - Set Text	input_Your Phone _	"0123456789"
7 - Set Encrypted Text	input_Password _	"ZT7alKdksLI="
8 - Set Encrypted Text	input_Confirm Password _	"ZT7alKdksLI="
9 - Click	input_patsub1	
10 - Click	a_list-home-list	
11 - Select Option By Value	select_Specialization	"General"; false
12 - Select Option By Value	select_Doctors	"ashok"; false
13 - Set Text	input_appdate	"2026-06-09"
14 - Select Option By Value	select_apptime	"14:00:00"; false
15 - Click	input_innuthn	

Figure 1.1: Katalon Studio Test Case Configuration

This image displays the Katalon Studio IDE capturing an automated test script for the patient registration and appointment booking workflow. Technical Details Inside: The built-in object repository has successfully captured the target web elements using relative locators. It shows a sequence of 15 automated commands , including Set Text for standard inputs (Name, Email, Phone) , Set Encrypted Text to securely handle passwords , and Select Option By Value to interact with the doctor and specialization dropdown menus.

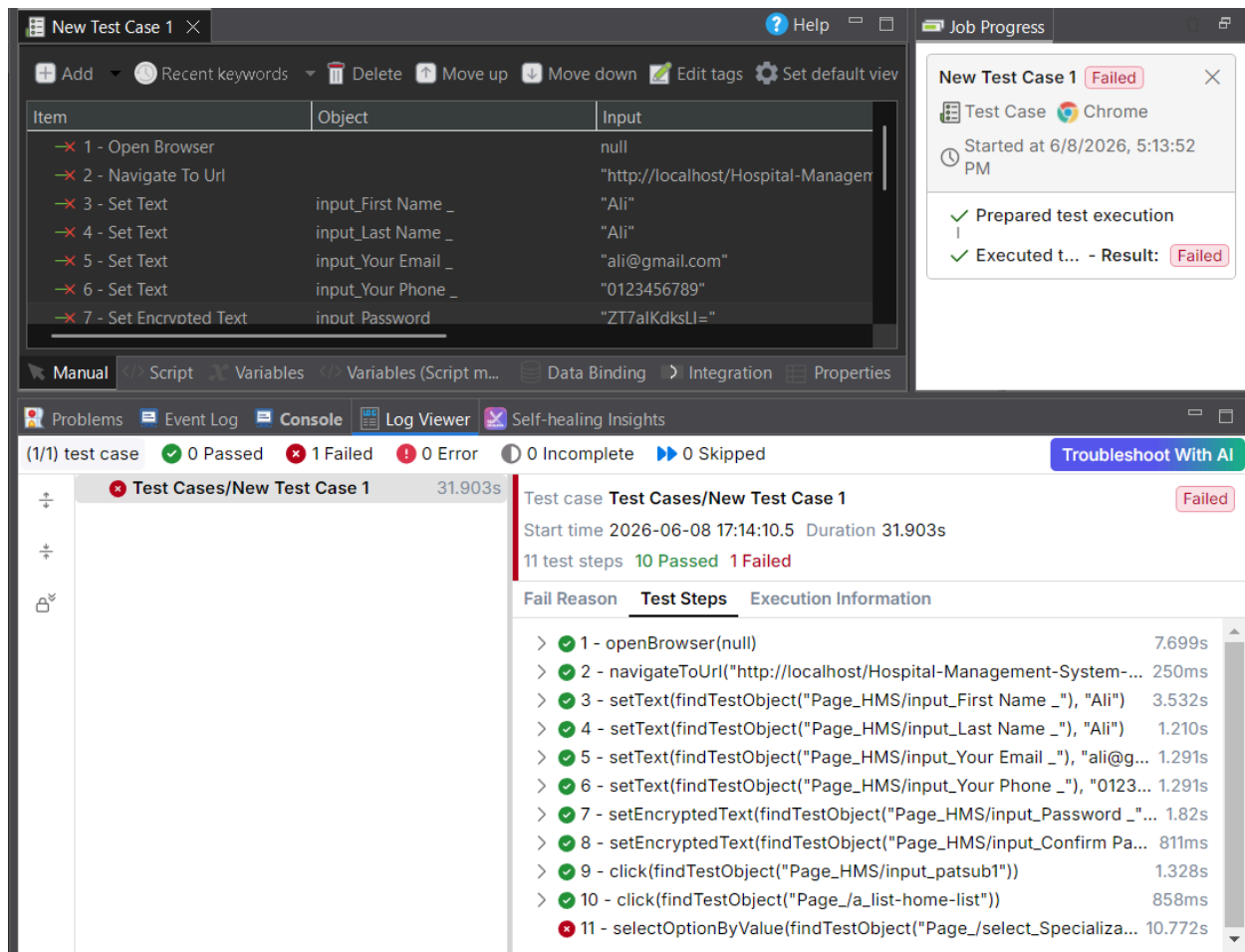


Figure 1.2: Katalon Studio Execution Log & Defect Tracking

This image demonstrates Katalon Studio's execution engine running the recorded test script and utilizing its Log Viewer to track the job progress. Technical Details Inside: The log shows the test ran for 31.903 seconds. It successfully highlights the verification capability of the tool: 10 steps passed successfully (marked in green) , while it accurately caught a failure on Step 11 (selectOptionByValue for Specialization). This proves the tool's effectiveness in identifying broken UI elements or missing dropdown values during automated runs.

Tool 2: Selenium IDE

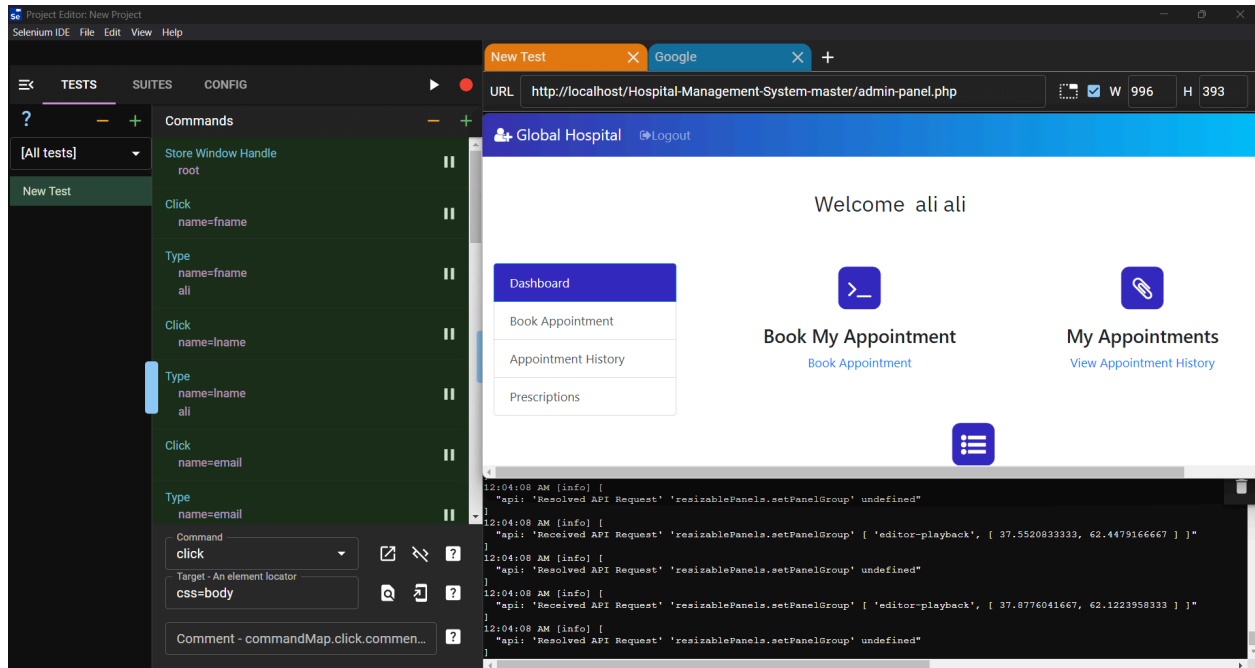


Figure 2.1: Selenium IDE Automated Execution

This image showcases the lightweight Selenium IDE browser extension actively executing a functional UI test against the HMS Admin Panel. Technical Details Inside: The IDE is running side-by-side with the local web browser. The command interface displays successful, sequential execution (highlighted in green) of UI interactions. It is successfully targeting elements using `name=fname` and `name=lname`, proving that the extension can seamlessly interact with the local XAMPP DOM environment without heavy backend configuration.

Tool 3: TestCase Studio

The screenshot displays the TestCase Studio application interface. At the top, there's a header bar with the title 'TestCase Studio' and a 'Set the TestCase Name here' input field. Below this is a toolbar with icons for Run, Pause, Stop, and other controls. The main area shows a table of test steps, each with a status icon, a description, data values, XPath locators, CSS selectors, Playwright commands, and a small screenshot. The steps are numbered 1 through 14, and all are marked as 'Done'. The bottom bar contains buttons for 'Test Case', 'Execute Steps in English as Code', 'Record Multi Test Cases', 'Jira | TestRail', and 'Add Columns'.

#	Steps	Data	Exp Result	XPath	CSS Selector	Playwright	Screenshots
1	Enter "ali" into "First Name "	ali		//input[@placeholder="First Name"]	input[placeholder="First Name"]	await page.locator("input[placeholder="First Name"]").fill("ali")	
2	Enter "ali" into "Last Name "	ali		//input[@placeholder="Last Name"]	input[placeholder="Last Name"]	await page.locator("input[placeholder="Last Name"]").fill("ali")	
3	Enter "ali@gmail.com" into "Your Email "	ali@gmail.com		//input[@placeholder="Your Email"]	input[placeholder="Your Email"]	await page.locator("input[placeholder="Your Email"]").fill("ali@gmail.com")	
4	Enter "0123456789" into "Your Phone "	0123456789		//input[@placeholder="Your Phone"]	input[placeholder="Your Phone"]	await page.locator("input[placeholder="Your Phone"]").fill("0123456789")	
5	Enter "*****" into "Password "	*****		//input[@id="password"]	#password	await page.locator("#password").fill("*****")	
6	Enter "*****" into "Confirm Password "	*****		//input[@id="cpassword"]	#cpassword	await page.locator("#cpassword").fill("*****")	
7	Click on "patsub1"			//input[@name="patsub1"]	input[value="Register"]	await page.locator("input[value="Register"]").click()	
8	Click on "Book Appointment"			//a[@id="list-home-list"]	#list-home-list	await page.locator("#list-home-list").click()	
9	select "General" from "Specialization:"	General		//select[@id="spec"]	#spec	await page.locator("#spec").selectOption("General")	
10	select "ashok" from "Doctors:"	ashok		//select[@id="doctor"]	#doctor	await page.locator("#doctor").selectOption("ashok")	
11	Enter "2026-06-10" into "appdate"	2026-06-10		//input[@name="appdate"]	input[name="appdate"]	await page.locator("input[name="appdate"]").fill("2026-06-10")	
12	select "8:00 AM" from "Select Time 8:00 AM 10:00 AM"	8:00 AM		//select[@id="apptime"]	#apptime	await page.locator("#apptime").selectOption("8:00 AM")	
13	Click on "bodyElement"			//body	body	await page.locator("body").click()	
14							

Done — 14 passed, 0 failed out of 14 steps.

Test Case Execute Steps in English as Code Record Multi Test Cases Jira | TestRail Add Columns

Figure 3.1: TestCase studio Automated Test Documentation

This image illustrates TestCase Studio operating as a test management utility, automatically documenting manual exploratory testing on the HMS registration and booking forms. Technical Details Inside: As the user interacts with the page, the tool translates the actions into 14 plain-English test steps (e.g., "Enter 'ali' into 'First Name'"). Crucially, it captures the exact Test Data used (e.g., 'ali@gmail.com', '2026-06-10') , extracts highly precise locators (XPath and CSS Selectors like `//input[@placeholder='First Name']`) , and attaches a micro-screenshot for every single step. This eliminates human error in manual test case writing.